
ParkFlow

Sistema de Gestión de Parqueadero Universitario

Informe Técnico — Entrega 3

Construcción de Software con Pruebas, CI/CD y DevSecOps

Diseño y Arquitectura de Software

Mayo 2026

Spring Boot 3.5 · TypeScript · PostgreSQL (Supabase) · Docker

JUnit · Newman · k6 · Cypress · GitHub Actions · OWASP · Gitleaks · Trivy

Tabla de Contenidos

1. Resumen Ejecutivo
2. Arquitectura del Sistema
3. Seguridad — Spring Security + JWT
4. Resiliencia — Circuit Breaker, Retry, Timeout
5. Pruebas Unitarias — JUnit + Mockito
6. Pruebas de API — Newman + Postman
7. Pruebas de Carga — k6
8. Pruebas de GUI — Cypress E2E
9. Pipeline CI/CD — GitHub Actions
10. DevSecOps — OWASP, Gitleaks, Trivy
11. Observabilidad — Prometheus, Grafana, Jaeger
12. Modelos 4+1 y C4
13. Retos y Soluciones

1. Resumen Ejecutivo

ParkFlow es un sistema de gestión de parqueadero universitario desarrollado como proyecto final de Diseño y Arquitectura de Software. La Entrega 3 implementa el ciclo completo de construcción de software: pruebas automatizadas en todos los niveles, pipeline CI/CD funcional, prácticas DevSecOps, y observabilidad en tiempo real.

Métrica	Valor	Detalle
Tests JUnit	20	20/20 passing
Assertions Newman	14	14/14 passing
Checks k6	400	50 VUs · 30s
Tests Cypress	14	4 suites E2E
Herramientas DevSecOps	3	OWASP · Gitleaks · Trivy
Endpoints REST	15+	13/15 protegidos con JWT

Estado del proyecto

Criterio	Estado	Detalle
Desarrollo del software	Completo	Backend Spring Boot + Frontend TypeScript
Pruebas unitarias + carga + API	Completo	JUnit 20/20, Newman 14/14, k6 400/400
Pruebas de GUI (Cypress)	Completo	4 suites, 14 casos de prueba E2E
CI/CD pipeline	Completo	GitHub Actions — pipeline verde
Docker (observabilidad)	Completo	Prometheus, Grafana, Jaeger en Docker Compose
DevSecOps	Completo	OWASP Dependency Check + Gitleaks + Trivy
Modelos 4+1 y C4	Completo	Documentados en docs/
Seguridad JWT	Completo	13/15 endpoints protegidos con JWT
Resiliencia	Completo	Circuit Breaker + Retry + Timeout

Stack tecnológico

Área	Tecnologías
Backend	Spring Boot 3.5 · Java 17 · Spring Security · JWT (jjwt 0.12.3) · Spring Data JPA · PostgreSQL (Supabase) · Micrometer · OpenTelemetry

Frontend	TypeScript puro · Vite · Sin frameworks · Fetch API con AbortController · localStorage para sesión · qrcode para QR
Pruebas	JUnit 5 + Mockito · Newman (Postman) · k6 · Cypress E2E · 4 suites de pruebas
CI/CD y DevSecOps	GitHub Actions · OWASP Dependency Check · Gitleaks · Trivy · Artifacts descargables
Observabilidad	Prometheus :9090 · Grafana :3001 · Jaeger :16686 · Docker Compose · OpenTelemetry OTLP gRPC
Base de datos	Supabase PostgreSQL · AWS us-east-2 · HikariCP pool · 4 tablas: users, spots, tickets, user_plates

2. Arquitectura del Sistema

ParkFlow sigue una **arquitectura orientada a servicios (SOA)** donde el backend actúa como servicio proveedor de una API REST y el frontend como servicio consumidor independiente.

Diagrama de arquitectura

```
Frontend (TypeScript + Vite) – puerto 3000
LoginPage | Dashboard | TicketPage | AdminPanel
|
| HTTP REST + JWT + AbortController (timeout)
v
Backend (Spring Boot 3.5 – Java 17) – puerto 8080
+-----+ +-----+ +-----+
| JwtFilter | | Controllers | | GlobalException |
| SecurityCfg |->| Auth|Spot|Ticket |->| Handler |
| JwtUtil | | Payment|User | +-----+
+-----+ +-----+
|
+-----v-----+
| ParkingFacade | <- Facade Pattern
+-----+
+-----+|+-----+
UserJpa SpotJpa TicketJpa
Repository Repository Repository
| | |
+-----+
v
Supabase PostgreSQL (Nube AWS us-east-2)
users | user_plates | spots | tickets

Observabilidad (Docker Compose):
Backend -> Prometheus:9090 -> Grafana:3001
Backend -> Jaeger:16686 (OTLP gRPC :4317)
```

Patrones de diseño aplicados

Patrón	Ubicación	Entrega
Singleton	ParkingManager	Entrega 1
Factory Method	VehicleFactory	Entrega 1
Strategy	PricingService, PaymentService	Entrega 1
Facade	ParkingFacade — orquesta toda la lógica de negocio	Entrega 2
Circuit Breaker	PaymentController — protege el servicio de pagos	Entrega 2

Repository	UserJpa, SpotJpa, TicketJpa repositories	Entrega 2
------------	---	-----------

Roles del sistema

Rol	Rutas	Capacidades
ADMIN	/admin/*	Panel global, CRUD celadores, estadísticas
ATTENDANT	/attendant/*	Registrar entradas, ver reportes, dar salidas
USER	/user/*	Ver ticket activo, pagar, gestionar placas con QR

3. Seguridad — Spring Security + JWT

ParkFlow implementa autenticación basada en **JSON Web Tokens (JWT)** con Spring Security, protegiendo 13 de 15 endpoints. Las contraseñas se almacenan con BCrypt — nunca en texto plano.

Flujo de autenticación

1. Usuario envía username + password → 2. Backend valida con BCrypt → 3. Genera JWT firmado HMAC-SHA256 → 4. Frontend guarda en localStorage → 5. Cada request envía JWT en header Authorization: Bearer → 6. JwtFilter valida firma y expiración antes de llegar al controller

Características del token JWT

Campo	Valor
Algoritmo de firma	HMAC-SHA256 (HS256)
Payload	username + role + iat (issued at) + exp (expiration)
Expiración	24 horas (86400000 ms)
Clave	Configurable vía application.properties — no hardcodeada
Almacenamiento frontend	localStorage del navegador

Endpoints protegidos

Endpoint	Requiere JWT	Rol mínimo
POST /api/auth/login	No	—
POST /api/auth/register	No	—
GET /api/spots	Sí	Cualquiera
POST /api/tickets	Sí	ATTENDANT
POST /api/payments	Sí	USER
DELETE /api/auth/workers/{u}	Sí	ADMIN
GET /api/users/me	Sí	Cualquiera
GET /actuator/prometheus	No	— (métricas públicas)

Verificación en vivo: Sin token → HTTP 403. Token de celador en endpoint de admin → HTTP 403. Token válido en endpoint correcto → HTTP 200.

4. Resiliencia — Circuit Breaker, Retry, Timeout

ParkFlow implementa tres patrones de resiliencia en el flujo de pagos — el punto más crítico del sistema — sin dependencias de librerías externas, lo que demuestra comprensión profunda de los patrones.

Circuit Breaker

Implementado manualmente en `PaymentController.java`. Tras 5 fallos consecutivos, abre el circuito y responde instantáneamente con HTTP 503 durante 30 segundos.

Parámetro	Valor
Umbral de fallos	5 fallos consecutivos (<code>FAILURE_THRESHOLD = 5</code>)
Tiempo en estado OPEN	30 segundos (<code>CIRCUIT_TIMEOUT = 30_000</code>)
Respuesta cuando OPEN	HTTP 503 con campo <code>retryAfterMs</code>
Recuperación	Automática — cierra solo tras 30 segundos

Retry con Backoff Exponencial

3 intentos automáticos con espera creciente. Si algún intento tiene éxito, resetea el contador del Circuit Breaker. Si todos fallan, incrementa el contador.

Intento	Espera	Acumulado
Intento 1	—	0s
Intento 2	1 segundo	1s
Intento 3	2 segundos	3s
Tiempo total máximo	4 segundos adicionales	~7s

Timeout

Capa	Endpoint	Timeout	Implementación
Frontend	GET <code>/api/spots/available</code>	2 segundos	<code>AbortController</code> en <code>api.ts</code>
Frontend	Todos los demás	10 segundos	<code>AbortController</code> en <code>api.ts</code>
Backend	Todos los endpoints	8 segundos	<code>spring.mvc.async.request-timeout</code>

5. Pruebas Unitarias — JUnit + Mockito

Se implementaron **20 pruebas unitarias** distribuidas en 3 clases, usando JUnit 5 y Mockito para aislar dependencias. Todas pasan con `./mvnw test`.

Resultado: Tests run: 20, Failures: 0, Errors: 0, Skipped: 0

JwtUtilTest — 6 tests

Test	Qué verifica
generateToken_debeRetornarTokenNoNulo	El token generado no es nulo ni vacío
extractUsername_debeRetornarUsernameCorrect	Extrae correctamente el username del payload
extractRole_debeRetornarRolCorrecto	Extrae correctamente el rol del payload
isTokenValid_tokenValido_debeRetornarTrue	Token válido retorna true
isTokenValid_tokenInvalido_debeRetornarFalse	Token con formato incorrecto retorna false
isTokenValid_tokenModificado_debeRetornarFalse	Token alterado falla la validación de firma

ParkingFacadeTest — 9 tests

Test	Qué verifica
admitVehicle_plazaLibre_debeCrearTicket	Crea ticket y marca plaza como ocupada
admitVehicle_plazaOcupada_debeLanzarExcepcion	Lanza <code>IllegalStateException</code> si plaza está ocupada
admitVehicle_plazaNoExiste_debeLanzarExcepcion	Lanza <code>IllegalArgumentException</code> si plaza no existe
exitAndPay_ticketActivo_debePagarYNoLiberarPlaza	Marca <code>paid=true</code> , <code>released=false</code> — plaza no se libera aún
exitAndPay_ticketYaPagado_debeRetornarTrueSinModificar	Idempotente — no modifica ticket ya pagado
exitAndPay_ticketNoExiste_debeLanzarExcepcion	Lanza <code>IllegalArgumentException</code> si ticket no existe
exitAndPay_calcularMonto_minimoCobra1Hora	Mínimo de cobro es 1 hora = \$3.000
releaseSpot_debeMarcarReleasedYLiberarPlaza	Marca <code>released=true</code> y plaza <code>occupied=false</code>
getActiveTickets_debeRetornarSoloNoPagados	Solo retorna tickets con <code>paid=false</code>

PaymentControllerTest — 4 tests

Test	Qué verifica
------	--------------

circuitBreaker_debeAbrirTras5Fallos	Tras 5 fallos consecutivos, responde 503
circuitBreaker_debeCerrarTras30Segundos	Circuito se cierra automáticamente
retry_debeIntentar3Veces	Se hacen exactamente 3 intentos antes de fallar
pago_exitoso_debeRetornarSuccess	Pago exitoso retorna status=SUCCESS

Uso de Mockito

```
@ExtendWith(MockitoExtension.class)
class ParkingFacadeTest {
    @Mock SpotJpaRepository spotJpaRepo;
    @Mock TicketJpaRepository ticketJpaRepo;
    @InjectMocks ParkingFacade facade;

    // Simula que ya hay plazas en BD para evitar @PostConstruct
    when(spotJpaRepo.count()).thenReturn(9L);
    when(spotJpaRepo.findAll()).thenReturn(List.of(spotLibre, spotOcupado));
}
```

6. Pruebas de API — Newman + Postman

Se creó una colección de Postman con **8 requests** y **14 assertions** que prueba el flujo completo del sistema con el backend real. Ejecutada con Newman en el pipeline CI/CD.

Resultado: 14/14 assertions passed — 0 failures

Flujo de prueba (orden de ejecución)

#	Request	Método	Assertions
01	Login Celador	POST /api/auth/login	Status 200 · Token recibido como string
02	Get Available Spot	GET /api/spots/available?type=CAR	Status 200 · Al menos un spot · Guarda spotId
03	Create Ticket	POST /api/tickets	Status 200 · Ticket tiene ID · Guarda ticketId
04	Get Active Tickets	GET /api/tickets/active	Status 200 · Respuesta es array
05	Process Payment	POST /api/payments	Status 200 · status = SUCCESS
06	Release Spot	POST /api/tickets/{id}/release	Status 200
07	Health Check	GET /actuator/health	Status 200 · status = UP
08	Sin Token 403	GET /api/spots	Status 403 — verifica seguridad

Variables dinámicas

El token JWT se obtiene en el request 01 y se usa automáticamente en todos los siguientes. El spotId se captura en el request 02 para crear el ticket en el 03. El ticketId se captura en el 03 para pagar en el 05 y liberar en el 06.

Comando de ejecución

```
newman run docs/parkflow-collection.json \  
--env-var "token=" \  
--env-var "ticketId="
```

Nota: Antes de cada ejecución limpiar en Supabase SQL Editor: UPDATE spots SET occupied = false; DELETE FROM tickets WHERE license_plate = 'NEWMAN1';

7. Pruebas de Carga — k6

Se implementó un script de prueba de carga con **k6** simulando 50 usuarios virtuales durante 30 segundos, ejecutando el flujo completo: login → crear ticket → procesar pago.

Resultado: 400/400 checks passed — 0 failures con 50 usuarios concurrentes

Configuración del test

Parámetro	Valor
Usuarios virtuales (VUs)	50 usuarios concurrentes
Duración	30 segundos
Archivo	docs/k6-load-test.js
Flujo probado	Login → GET spots → POST ticket → POST payment

Checks implementados

Check	Condición
Login exitoso	HTTP 200 + token en respuesta
Spots disponibles	HTTP 200 + array no vacío
Ticket creado	HTTP 200 + id en respuesta
Pago procesado	HTTP 200 + status = SUCCESS

Comando de ejecución

```
k6 run docs/k6-load-test.js
```

8. Pruebas de GUI — Cypress E2E

Se implementaron pruebas end-to-end con **Cypress** cubriendo los 3 roles del sistema y los flujos principales de navegación, autenticación y control de acceso.

Resultado: 4 suites · 14 casos de prueba E2E

Suites de pruebas

Archivo	Tests	Qué cubre
login.cy.ts	4	Login exitoso celador/admin/usuario · Login fallido muestra error
attendant.cy.ts	4	Dashboard stats · Sidebar por rol · Navegación · Sin opciones admin
user.cy.ts	4	Dashboard · Sección placas · Formulario agregar placa · Ticket y Pago
admin.cy.ts	4	Panel admin · Trabajadores · Bloqueo rutas de otros roles · Logout

Casos de prueba

- ✓ Login exitoso como celador redirige a /attendant/dashboard
- ✓ Login exitoso como admin redirige a /admin/panel
- ✓ Login exitoso como usuario redirige a /user/dashboard
- ✓ Login fallido muestra mensaje de error visible
- ✓ Dashboard celador muestra estadísticas de plazas
- ✓ Sidebar celador NO muestra opciones de admin
- ✓ Celador navega correctamente a Registrar Entrada
- ✓ Celador navega correctamente a Reportes
- ✓ Usuario ve sección de vehículos registrados
- ✓ Usuario puede abrir formulario para agregar placa
- ✓ Sidebar usuario NO muestra opciones de celador ni admin
- ✓ Admin navega a gestión de trabajadores
- ✓ Admin no puede acceder a rutas de usuario
- ✓ Cierre de sesión funciona en todos los roles

Configuración

```
// cypress.config.ts
export default defineConfig({
```

```
e2e: {  
  baseUrl: 'http://localhost:3000',  
  specPattern: 'cypress/e2e/**/*.cy.ts',  
  defaultCommandTimeout: 8000,  
  viewportWidth: 1280,  
  viewportHeight: 720,  
  video: false  
}  
});
```

9. Pipeline CI/CD — GitHub Actions

Se configuró un pipeline completo en GitHub Actions que se ejecuta automáticamente en cada push a la rama main. El pipeline incluye compilación, pruebas, análisis de seguridad y generación de artifacts.

Estado: Pipeline verde — todos los pasos pasan exitosamente

Pasos del pipeline

Paso	Herramienta	Qué verifica
1. Compilar backend	Maven	mvnw clean compile — sin errores de compilación
2. Pruebas unitarias	JUnit + Mockito	20/20 tests pasan
3. Backend en background	Spring Boot	Espera /actuator/health = UP
4. Pruebas de API	Newman	14/14 assertions con backend real
5. OWASP Dependency Check	Maven plugin	CVEs en dependencias Maven
6. Gitleaks	GitHub Action	Secrets en historial de git
7. Trivy	GitHub Action	Vulnerabilidades en filesystem
8. Package JAR	Maven	Genera artifact descargable

Ubicación del pipeline

```
.github/workflows/ci-cd.yml
# Para disparar manualmente:
# GitHub -> Actions -> ParkFlow CI/CD -> Run workflow
# URL del repositorio:
# https://github.com/AnaSofia64/Proyecto-Disenio/actions
```

Artifacts generados

Cada ejecución del pipeline genera artifacts descargables desde GitHub Actions: el JAR compilado del backend, el reporte HTML de OWASP Dependency Check, y los resultados de Newman en formato JSON.

10. DevSecOps — OWASP, Gitleaks, Trivy

Se integraron tres herramientas de seguridad en el pipeline CI/CD, cubriendo análisis de dependencias, detección de secretos y escaneo de vulnerabilidades en el sistema de archivos.

OWASP Dependency Check

Analiza las dependencias Maven del backend buscando CVEs conocidos en la base de datos NVD. Genera reporte HTML detallado.

Campo	Valor
Herramienta	OWASP Dependency Check Maven Plugin
Umbral académico	CVSS = 11 (nunca bloquea — solo reporta)
Umbral producción recomendado	CVSS = 7 (bloquea el build)
Reporte generado	target/dependency-check/dependency-check-report.html
Comando	mvnw org.owasp:dependency-check-maven:check

Gitleaks — Detección de Secretos

Escanea el historial completo de git buscando secretos expuestos como API keys, passwords, tokens o credenciales hardcodeadas en el código fuente.

Campo	Valor
Herramienta	Gitleaks GitHub Action
Resultado esperado	no leaks found
Qué detecta	API keys, passwords, JWT secrets, tokens de acceso
Comando local	gitleaks detect --redact -v

Trivy — Escaneo de Vulnerabilidades

Escanea el filesystem del proyecto buscando vulnerabilidades conocidas en las dependencias y librerías usadas.

Campo	Valor
Herramienta	Trivy GitHub Action (aquasecurity)
Resultado típico	10 CVEs reportados — 3 CRITICAL, 7 HIGH en Tomcat/Spring Boot

Exit code	0 — solo reporta, no bloquea el build
Comando local	trivy fs Back-end/backend

Nota sobre resultados: Los CVEs reportados por Trivy corresponden a versiones de Tomcat y Spring Boot incluidas por defecto. En un entorno de producción se actualizarían las dependencias o se aplicarían mitigaciones. Para el contexto académico, el umbral está configurado para reportar sin bloquear.

12. Modelos 4+1 y C4

Modelo 4+1 — Resumen de vistas

Vista	Descripción	Elementos clave
Lógica	Funcionalidad desde perspectiva del usuario	Controllers, ParkingFacade, Roles, Entidades JPA
Procesos	Procesos en tiempo de ejecución	Flujo de login, flujo de pago con CB, registro de entrada
Desarrollo	Organización del código fuente	Paquetes por capa, patrones aplicados por entrega
Física	Despliegue en infraestructura	Localhost, Supabase nube, Docker Compose, GitHub Actions
Escenarios	Casos de uso que validan las otras vistas	Usuario paga, servicio falla, celador da reportes, admin gestiona

Modelo C4 — Niveles

Nivel	Muestra
Nivel 1 — Contexto	ParkFlow interactúa con Usuario, Celador, Admin + Supabase y Stripe
Nivel 2 — Contenedores	Frontend, Backend, Supabase PostgreSQL, Stack Docker de observabilidad
Nivel 3 — Backend	Seguridad, Controllers, ParkingFacade, Repositorios JPA, Observabilidad
Nivel 3 — Frontend	router.ts, auth.ts, api.ts, Pages por rol, Components compartidos

Los diagramas visuales completos se encuentran en docs/diagrams/ y los documentos detallados en docs/modelo-4+1.md, docs/modelo-c4.md y docs/diagramas-c4-4plus1.html.

13. Retos y Soluciones

Reto	Solución implementada
Límite de conexiones en Supabase free tier (MaxClientsInSessionMode)	Configurar HikariCP con maximum-pool-size=2 y agregar prepareThreshold=0 en la URL JDBC para deshabilitar prepared statements del pooler
Hibernate preparaba DDL con ddl-auto=update agotando conexiones	Cambiar a ddl-auto=none y ejecutar migraciones manuales en Supabase SQL Editor
Circuit Breaker con throw — "unreachable statement" no compilaba en Java	Usar if(true) throw para engañar al compilador durante demos sin alterar comportamiento real
Tickets pagados desaparecían de reportes antes de que el celador diera salida	Agregar campo released a TicketEntity — separar estado de pago del estado de salida física de la plaza
user_plates necesitaba vehicle_type pero era un ElementCollection simple	Migrar a entidad UserPlateEntity con su propia tabla, ID y repositorio JPA independiente
Newman en el pipeline necesitaba el backend corriendo en localhost	Iniciar Spring Boot en background en GitHub Actions y esperar con curl al endpoint /actuator/health antes de ejecutar Newman
CORS bloqueaba requests del frontend al backend	Configurar CorsConfig.java con los orígenes permitidos (localhost:3000, 5173, 5174)
Frontend TypeScript puro sin React dificulta el manejo de estado	Patrón de clases con métodos de renderizado y re-renderizado explícito por componente (buildCard, buildTable)
Tests JUnit necesitaban aislar @PostConstruct de ParkingFacade	Mockear spotJpaRepo.count() para retornar 9L y evitar que el init() intente crear plazas en BD